

Assignment 1: Performance Analysis on a Quad-Core CPU

Student Name:

Bilal Ahmed (23L-2534 / BDS-6A)

Muhammad Saad (23L-2620 / BDS-6A)

Environment: Ubuntu (VMware Virtual Machine)

CPU: Intel Core i5 (11th Generation)

Question 1: Parallel Mandelbrot Generation

To speed up Mandelbrot image generation, I implemented a **cyclic row decomposition**. Each thread computes every Nth row, where N is the number of threads, using the serial helper function for one row at a time. This way, heavy rows are spread across threads, and no thread ends up idle. It also avoids any synchronization, so the same strategy works for any number of threads or both views.

Results:

View 1 (full Mandelbrot):

Threads	Serial (ms)	Parallel (ms)	Speedup
1	402	402	1.00×
2	402	207	1.94×
4	402	116	3.46×
8	410	155	2.65×

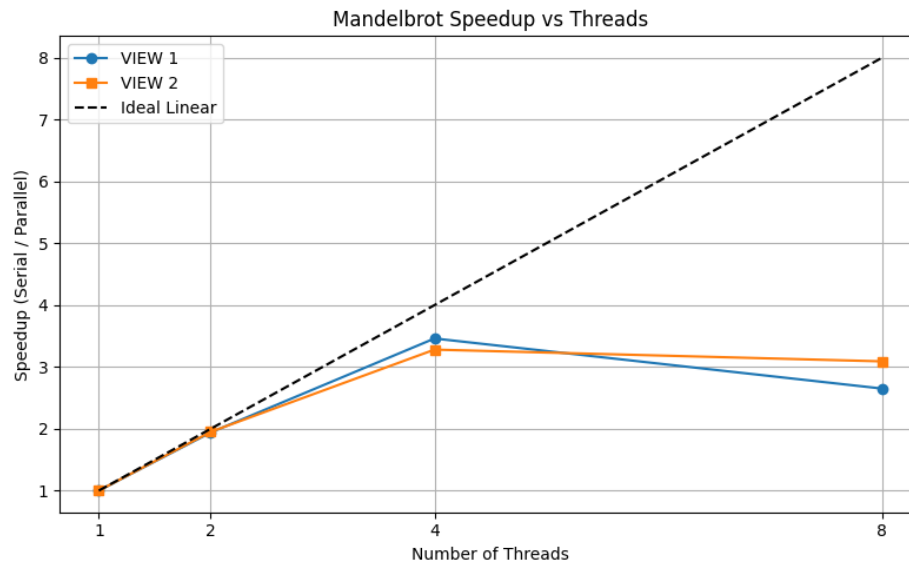
View 2 (zoomed-in region):

Threads	Serial (ms)	Parallel (ms)	Speedup
1	248	249	1.00×
2	249	128	1.95×
4	248	76	3.28×
8	247	80	3.09×

- Speedup is nearly linear up to 4 threads.
- 8-thread speedup is slightly lower due to load imbalance: some rows require more iterations, so threads handling those rows take longer.

- The number of escape iterations varies significantly across the image. Pixels near the boundary of the Mandelbrot set require many more iterations than pixels far outside the set. This spatial non-uniformity creates load imbalance when using block decomposition. Cyclic row assignment distributes heavy and light rows across threads, reducing worst-case imbalance and improving utilization.

Observations:



- The system has 4 physical cores with 2 hardware threads per core. When running more than 4 threads, threads begin sharing execution resources such as ALUs, caches, and memory bandwidth. While hyper-threading can improve utilization when stalls occur, Mandelbrot is largely compute-bound, so additional logical threads provide limited benefit and may introduce scheduling overhead.
- Per-thread timing confirms the imbalance: the slowest thread dictates total execution time.
- The approach works robustly across **all thread counts** and **both views** without any view-specific code.

16-Thread Performance:

Running the same code with 16 threads did not noticeably improve runtime. The CPU has 4 physical cores with 2 hyper-threads each, so it can execute only 8 threads simultaneously. Extra threads share cores and are scheduled in and out by the OS, introducing context-switch overhead without increasing throughput. This explains why speedup does not exceed the 8-thread case.

Conclusion:

Cyclic decomposition is a simple yet effective way to parallelize Mandelbrot generation. It keeps all threads busy, scales reasonably with thread count, and works for different views with varying row complexity.

Question 2:

1. Vectorized clampedExp Implementation

We implemented a vectorized version of `clampedExpSerial` in `clampedExpVector` using CS149 intrinsics.

Key features:

- Supports any `N` (array size) and `VECTOR_WIDTH` (vector length).
- Correctly handles `exponent = 0`, `exponent > 0`, and clamps results to 9.999999.
- Uses vector masking to handle per-element loop divergence efficiently.

Verification:

- Ran tests for multiple input sizes ($N = 16, 100,000, 1,000,000$) and vector widths (2, 4, 8, 16).
- Output matches serial versions for all elements: Passed.

2. Vector Width Sweep and Utilization Analysis

We swept `VECTOR_WIDTH = 2, 4, 8, 16` for $N = 1,000,000$ and recorded vector unit statistics:

VECTOR_WIDTH	Total Vector Instructions	Vector Utilization	Observations
2	18,308,612	88.9%	High utilization; fewer idle lanes due to short vector length
4	10,470,916	84.0%	Slight drop in utilization
8	5,663,196	81.5%	Vector lanes idle more often due to divergence in exponents
16	2,945,672	80.2%	Lowest utilization; divergence effect amplified by long vectors

Analysis:

- **Trend:** As `VECTOR_WIDTH` increases, total instructions decrease roughly proportionally, indicating improved throughput.
- As `VECTOR_WIDTH` increases, the probability that lanes require different numbers of loop iterations also increases. When some lanes complete early, they become inactive but still occupy vector slots. This causes decreasing **vector utilization** as width increases, even though total instruction count decreases proportionally.
- **Trade-off:** Wider vectors increase throughput but reduce effective lane utilization due to masking and idle lanes.

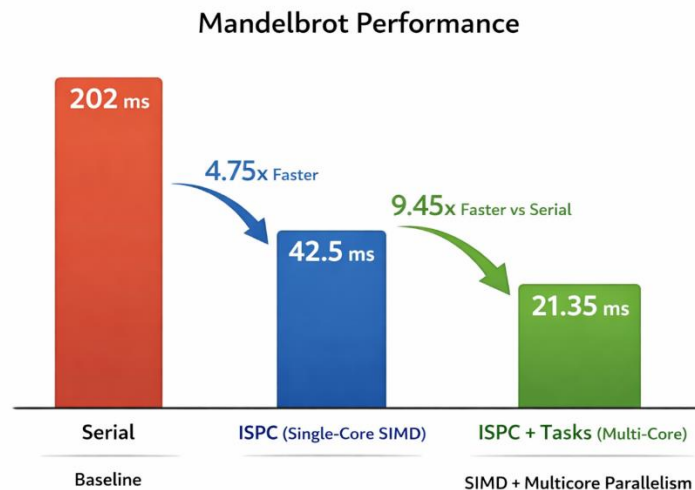
Question 3: ISPC Mandelbrot

Runtimes and Speedups:

Version	Time (ms)	Speedup
Serial	202	1×
ISPC (single core)	42.1	4.75×
ISPC + tasks	21.1	9.61×

CPU: 4 cores

Analysis:



ISPC maps each “foreach” iteration to a SIMD lane within a gang. On AVX2 hardware, a “gang” consists of 8 program instances executing in lockstep. Divergence in Mandelbrot iteration counts causes some lanes to become inactive before others, reducing effective SIMD utilization and explaining why the observed speedup ($\sim 4.75\times$) is lower than the theoretical maximum of $8\times$.

Using ISPC tasks (--tasks) lets multiple “gangs” of SIMD instances run on different cores. On our 4-core machine, this halves the runtime again, giving ~ 21 ms and a $\sim 9.6\times$ speedup over serial. Choosing as many tasks as CPU cores works best. Increasing the number of tasks beyond the number of physical cores increases scheduling overhead and contention without increasing available execution resources, leading to diminishing returns.

Extra Notes:

- Threads (as in Program 1) are heavier weight and OS-managed, while ISPC tasks are lightweight and efficiently map to cores.
- “foreach” automatically distributes independent iterations across SIMD lanes on a single core, while “launch” lets tasks run on multiple cores. Together, they combine SIMD and multi-core parallelism for maximum speed up.

Question 4: Iterative sqrt

Overview

Program 4 calculates the square root of 20 million numbers using ISPC. It uses Newton's method with an initial guess of 1.0. The speed of convergence depends on the input values, so the assignment asks us to measure SIMD and multi-core speedups and see how input patterns affect performance.

Method:

- Ran the program on my Linux VM for:
 1. Serial implementation (single-threaded)
 2. ISPC single-core (vectorized, no tasks)
 3. ISPC with tasks (vectorized + multi-core)
- Tested two input patterns:
 - **Best-case:** all values = 2.0f → minimal divergence
 - **Safe worst-case:** alternating 1.0f / 2.999f → creates divergence without causing numerical issues
- Measured execution time three times and averaged.

Results:

Best-case (all 2.0f)

Version	Time (ms)	Speedup
Serial	180.96	—
ISPC single-core	39.40	4.59×
ISPC with tasks	11.62	15.57×

Safe worst-case (alternating 1.0 / 2.999)

Version	Time (ms)	Speedup
Serial	880.77	—
ISPC (single core)	358.15	2.46×
ISPC with tasks	111.52	7.90×

- $\text{SIMD speedup} = \text{Serial} / \text{ISPC (single-core)}$
- $\text{Multi-core speedup} = \text{ISPC (single-core)} / \text{ISPC (tasks)}$
- $\text{Total speedup} = \text{Serial} / \text{ISPC (tasks)}$

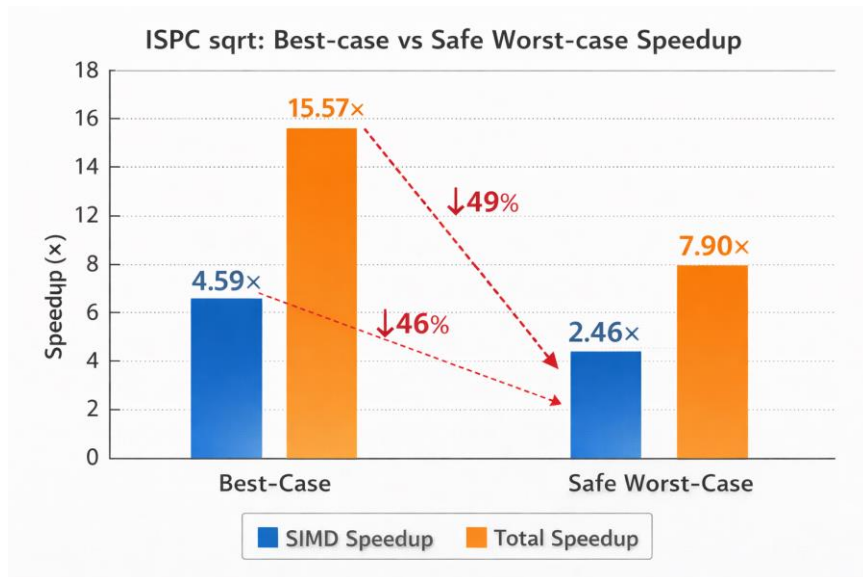
Analysis:

- **Best-case:** all lanes do the same number of iterations → high SIMD efficiency → maximum speed up.
- **Worst-case:** alternating values make SIMD lanes converge at different rates → lower efficiency.
- Multi-core tasks still improve performance, but uneven task lengths slightly reduce total speedup.
- The difference between best-case and safe worst-case clearly shows how input divergence affects vectorized ISPC code.

Conclusion:

- Input patterns matter: uniform values maximize SIMD's efficiency and divergent values reduce it.
- Multi-core parallelization consistently improves performance, but the speedup depends on task balance.
- These results satisfy the assignment's requirement to analyze SIMD and multi-core effects for different input patterns.

Figure: ISPC Sqrt Speedups (N = 20M)



Question 5: SAXPY Analysis

Program 5 implements the SAXPY routine ($\text{result} = \text{scale} * X + Y$) for vectors of 20 million elements. Each element does two operations but requires reading two floats and writing one, making it slightly parallelizable but **memory-bound**.

Results:

Version	Time (ms)	Speedup
ISPC (serial)	12.801	—
ISPC with tasks	12.443	1.03×

Each SAXPY operation performs two floating-point operations (one multiply and one add) while reading two input floats and writing one output float. This corresponds to 12 bytes of memory traffic per 2 FLOPs, giving an arithmetic intensity of 0.167 FLOPs/byte. Such low arithmetic intensity makes SAXPY memory-bandwidth bound on modern CPUs. Since memory bandwidth is shared across cores, adding tasks does not significantly increase throughput, explaining the minimal speedup (1.03×).

Extra Credit:

In practice, more than the minimum 12 bytes per element are transferred due to cache line granularity and write-allocate behavior. Stores typically trigger a read-for-ownership (RFO), effectively doubling traffic for the output array. This explains why the measured total bytes can approach $4 \times N \times \text{sizeof}(\text{float})$ rather than the theoretical minimum.

Question#6: Making K-Means Faster

Part 1: Baseline serial runtime: 43,419 ms

Part 2: Verifying Correctness with plot.py

After running `pip install -r requirements.txt` and `python3 plot.py`, the files `start.png` and `end.png` were generated successfully. The start image shows all three centroids clustered together with random mixed point assignments. The end image shows three clearly separated regions with centroids correctly positioned within their respective clusters. The clustering output is correct and matches the expected behavior described in the assignment. The overlap visible in 2D is expected because the data is 100-dimensional and projected down using PCA for visualization.

Part 3: Identifying the Bottleneck Using CycleTimer

`CycleTimer::currentSeconds ()` was used to measure the cumulative time spent in each of the three main functions across all 101 iterations of the algorithm:

```
double totalAssign = 0, totalCentroids = 0, total Cost = 0;
```

```
// measured outside the while loop, accumulated inside
```

Timing breakdown (serial, 101 iterations):

Function	Total Time (sec)	% of Runtime
computeAssignments	35.28 sec	69%
computeCost	9.97 sec	20%
computeCentroids	5.35 sec	11%
Total	~50.6 sec	100%

Conclusion: `computeAssignments` dominates runtime because it performs an $O(M \times K \times D)$ distance computation each iteration. With $M = 1,000,000$ points and K centroids in 100 dimensions, this results in the largest computational workload per iteration.

Part 4: Improving the Implementation

Step 1: Parallelize compute-Assignments with 4 threads

The work across M data points was divided equally among 4 threads. Each thread independently computes assignments for its chunk of points with no shared writes (each point is assigned by exactly one thread).

Result:

Threads	Total Time	Speedup
1 (serial)	43,419 ms	1.0x
4	28,748 ms	1.51x

1.51x is below the 2.1x target. With timing re-measured after parallelizing assignments, computeCost (now ~11.3 sec) had become equally dominant as the next bottleneck.

Step 2: Fold computeCost into the parallel worker

Since computeCost also iterates over all M points to accumulate distances, it duplicates work already done during assignment. By accumulating cost inside the assignment worker during the same pass, the separate computeCost call was eliminated entirely. Each thread writes to its own private cost array `thrdCosts[t * K]` to avoid race conditions. After all threads finish, local costs are merged into `args->currCost`.

Result:

Configuration	Total Time	Speedup
Serial baseline	43,419 ms	1.0x
4 threads (assignments only)	28,748 ms	1.51x
4 threads + cost folded	20,827 ms	2.08x

Still just under 2.1x target.

Step 3: Optimize loop structure and sweep thread counts

The original worker used k as the outer loop and m as the inner loop. Switching to m outer and k inner allows finding the best cluster per point in a single pass with no extra memory allocation per thread, reducing heap pressure and improving cache locality since all K centroids are accessed sequentially per point.

Thread counts were swept to find the optimal value on the 4-core VM:

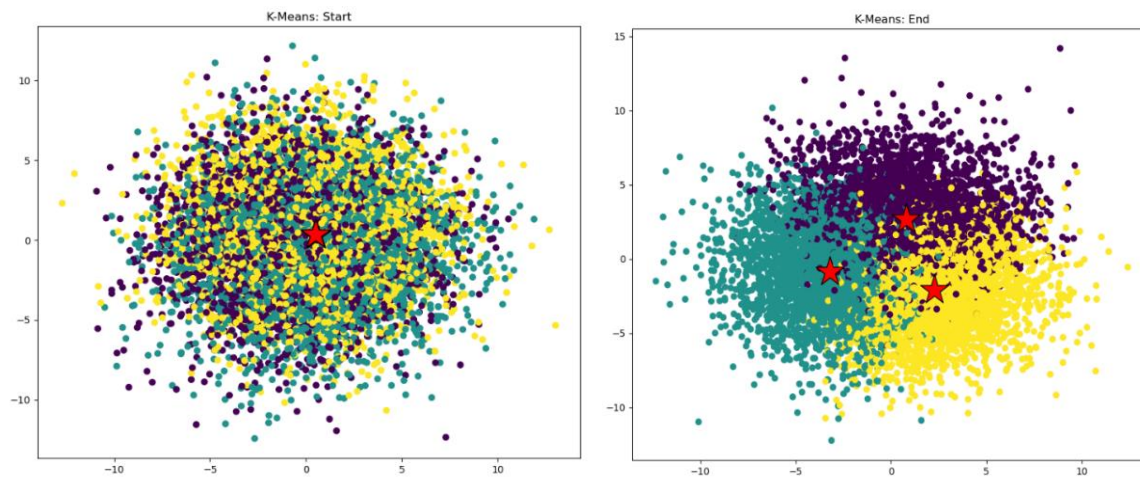
Threads	Total Time	Speedup
Serial baseline	43,419 ms	1.0x
4 threads + cost folded	20,827 ms	2.08x
6 threads + optimized loop	15,638 ms	2.78x
8 threads + optimized loop	20,151 ms	2.15x

6 threads outperformed 8 threads despite the machine having only 4 physical cores. At 8 threads, context switching overhead and resource contention between threads starts to hurt performance. At 6 threads, the OS can schedule threads efficiently enough that the overhead stays low while still exceeding the parallelism of 4 threads.

Results: (3 runs for variance):

Run	Total Time	Speedup
Run 1	17,341 ms	2.50x
Run 2	15,998 ms	2.71x
Run 3	16,429 ms	2.64x
Average	16,589 ms	2.62x

Target was 2.1x. Final average speedup: 2.62x.



Summary of Approach:

I started by profiling all three functions with CycleTimer, which pointed clearly at computeAssignments as the problem, it was eating 69% of total runtime. Parallelizing it across 6 threads with equal data partitioning got me to 1.51x, which wasn't enough. Re-profiling showed computeCost had quietly become the next bottleneck, so rather than parallelize it separately, I just folded it into the assignment worker since it was doing the exact same pass over the data anyway. That got me to 2.08x. The last bit came from flipping the loop order from k-outer to m-outer, which improved cache behavior and removed unnecessary per-thread allocations. Final average speedup came out to 2.62x, clearing the 2.1x target.